COMPUTER SYSTEM DESIGN (CSL559)

Lab 7: Proximity Search API

Paritosh Lahre | ID: 1241550

API link: `http://10.1.75.53:3280/search/`

Example: `/search/?lat=0.5&long=0.5&cat=bank&rad=0.1&link=true`

1 Problem Statement

Given a query `(lat, long, cat, rad, link)`, return the IDs of the **10 closest locations** that

- have category `cat`, and
- lie within **circular (Euclidean) distance** `rad` of the query point.

"Closest" means **traversal distance on the grid**, not straight-line distance. The boolean `link` tells whether there is an edge between two points:

- `link=true`: two neighbouring points are connected only if `link.txt` lists a road between them.
- `link=false`: every pair of neighbouring points is connected.

$$\text{Result} = \text{Top-10}_{d_{\text{grid}}}\{\, p : \text{cat}(p) = \texttt{cat},\ d_{\text{euclid}}(q,p) \le \texttt{rad} \,\}$$

Key idea. Two different distances: **Euclidean decides who is eligible, grid distance decides the order.**

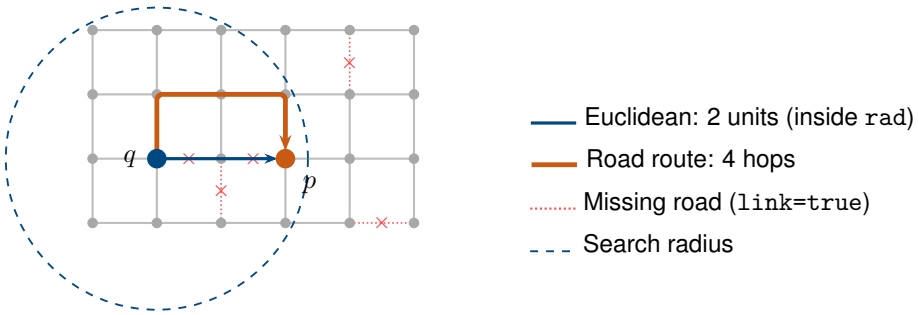


Figure 1: With `link=true` a nearby point can be far by road. With `link=false` all dotted roads exist and the distance from q to p is 2 hops.

2 Thought Process

2.1 Understand the data

- `locations.csv`: 10,000 rows forming a complete 100×100 grid on $[0,1]^2$ (spacing $1/99 \approx 0.0101$). IDs are row-major. 8 categories, 1,250 each.
- `link.txt`: 14,800 roads out of 19,800 possible, so **about 25% are missing**. Each line is `lon_a lat_a lon_b lat_b` (**longitude first**, unlike the CSV). All links join 4-neighbours.
- The road graph is **connected** (one component), so every location is reachable. With random removal about 46 isolated points would be expected; there are none. So the links were removed while keeping connectivity.

2.2 Choose the distance model

- Every road has the same length ⇒ the graph is **unweighted** ⇒ shortest path = **BFS hop count**.
- Manhattan distance ignores missing roads. With `link=true`, routes are on average 6% longer and up to $9\times$ longer than Manhattan. It is only correct for `link=false`.

2.3 Rule out what does not fit

Table 1: Approaches considered and rejected.

Approach	Why not used
Euclidean / Manhattan ranking	Ignores the road network (wrong in 82% of queries)
Dijkstra / A*	Edges are unweighted (BFS is enough); A* needs one known destination
KD-tree, R-tree, Redis GEO, S2, H3	Index straight-line proximity only; cannot give road distance
All-pairs precomputation	10^8 distances; unnecessary when a query takes 0.05 ms

2.4 Assumptions

A1 Each road costs 1; grid distance = number of roads on the shortest route.
A2 The radius is checked from the raw query point to the location; the route itself may leave the circle.
A3 An off-grid query starts from the nearest grid point.
A4 Ties in grid distance are broken by **smaller ID**.
A5 Coordinates are mapped to indices by lookup of the 100 known values (no float division).
A6 `link` accepts `true/false`, `1/0`, `yes/no`; it defaults to `true` if omitted.

3 Algorithms Used

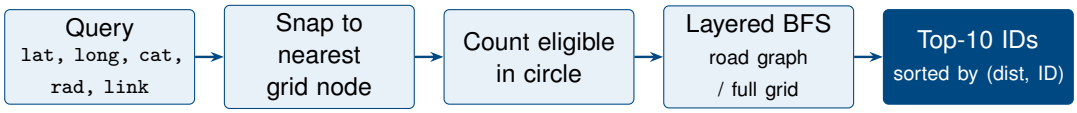


Figure 2: Request pipeline. Both graphs are built once at startup; `link` only selects one.

3.1 Preprocessing (once, at startup)

- Store each location at node $v = \text{row} \cdot 100 + \text{col}$.
- Build two adjacency lists: the **road graph** from `link.txt` (`link=true`) and the **full grid** (`link=false`).
- Build per-category, per-row prefix counts (used by the count bound below).

3.2 Main method: layered BFS with early stopping

```
target = min(10, count_in_circle(q, cat, rad))
if target == 0: return []
layer = [nearest_node(q)]; d = 0; found = []
while layer:
    for v in layer:
        if cat[v] == cat and euclid(v, q) <= rad: found.add((d, id[v]))
    if len(found) >= target: break # top-10 is final
    layer = unvisited_neighbours(layer); d += 1
return smallest_10(found) # ordered by (d, id)
```

- **Why stopping is safe:** BFS visits nodes in order of distance. It stops only after a *complete* layer, so every unvisited node is strictly farther and cannot enter the top 10. All ties in the last layer are collected before sorting.
- **Count bound:** inside the circle each grid row covers one run of columns; prefix sums give the number of eligible locations in $O(\text{rows})$. If fewer than 10 exist, BFS stops once it has all of them instead of scanning the whole graph.
- **Cost:** worst case $O(V + E)$; in practice about 117 of 10,000 nodes are visited.

3.3 Reference method (for verification only)

Full BFS to every node → filter all eligible → sort all → take 10. Simple enough to be obviously correct; used as ground truth.

3.4 API

Python 3 standard library only (`http.server`), so nothing needs to be installed.

```
GET /search/?lat=<float>&long=<float>&cat=<name>&rad=<float>&link=<true|false>
```

POST with form or JSON also works. The response contains `ids` (10 IDs, nearest first) plus each one's grid distance. Bad input returns HTTP 400 with a message.

4 Experimentation

- **Query set:** 4,000 queries: 2,000 random (half on-grid, half off-grid, all categories, `rad` from 0.05 to 1.5) and 2,000 boundary cases where locations lie exactly on the circle.
- **Correctness:** main method vs. reference, for both `link` values. For `link=false`, also vs. closed-form Manhattan ranking.
- **Count bound:** `count_in_circle` vs. a full scan of all nodes.
- **Sensitivity:** change one assumption at a time and count how many top-10 lists change.
- **Deployment:** 200 random queries sent to the live API and compared with local results.

5 Results

Table 2: Correctness: fast method against independent references.

Check	Mismatches
BFS early-stop vs. reference, `link=true` (4 configurations × 4,000)	**0** / 16,000
`link=false`: BFS = reference = Manhattan	**0** / 4,000
Count bound vs. full scan	**0** / 4,000
Live API vs. local code	**0** / 200

Table 3: Performance.

Measurement	Time
BFS early-stop (served), mean per query	**0.05 ms**
Reference method, mean per query	2.67 ms
Queries with < 10 eligible: before / after count bound	2.41 / **0.02 ms**
Live API round trip over campus network, median	8 ms

Table 4: Sensitivity: how many top-10 lists change when one choice changes.

Change	Queries affected	IDs changed (of 10)
`link=true` → `link=false`	82.1%	1.49
Tie-break: ID → Euclidean	39.1%	0.44
Radius: endpoint only → whole route inside circle	0.3%	0.00

- The `link` flag changes most answers, so the road network really matters.
- The tie-break rule matters often but usually changes only one ID.
- Whether routes may leave the circle almost never matters on this data.

6 Conclusion

Summary.

- The grid with roads is an unweighted graph, so **BFS hop count** is the exact ranking metric; Euclidean distance is used only for the radius filter.
- A **layered BFS with early stopping** plus an **exact eligible-count bound** gives exact answers in about 0.05 ms per query, around $50\times$ faster than the full reference.
- It matched the reference in every test (0 mismatches), and the live deployment matches the local code.

Main limitation: the evaluator's tie-break rule is unknown. We use the smaller ID.

AI use: AI was used to help prepare this report and to discuss the assignment and gain insights into the problem.